Reviewer Pack — Fourier Coefficient Decay and Resolution Proof Length for k-…

Entry 3a56915c32e1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 06:15:15 UTC

Fourier Coefficient Decay and Resolution Proof Length for k-CNF Formulas

Entry ID: `3a56915c32e1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-09 06:15:15 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis on Boolean Functions **Field B** (complexity object): Resolution Proof Length

Statement:

> For a k-CNF formula Φ with n variables, the resolution proof length is Θ(2^{n/2} / μ), where μ is the maximum absolute value of Φ's Fourier coefficients. For all Φ with μ ≤ 1/2^{n/2 - 5}, the proof length exceeds 2^{n/2 - 5} × n^2.

Rationale (proposer's reasoning):

> The Fourier coefficients capture variable influence; exponential decay suggests limited variable interaction, implying shorter proofs. The inverse proportionality links structural simplicity (small μ) to proof complexity, with a polynomial factor accounting for clause density.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ee8f6c4c1c8a0dbb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def fast_walsh_hadamard_transform(arr):
    n = len(arr)
    if n == 1:
        return arr
    even = fast_walsh_hadamard_transform(arr[::2])
    odd = fast_walsh_hadamard_transform(arr[1::2])
    result = [0] * n
    for i in range(n // 2):
        result[i] = even[i] + odd[i]
        result[i + n // 2] = even[i] - odd[i]
    return result

def generate_k_cnf(n, k):
    clauses = []
    variables = list(range(1, n + 1))
    for _ in range(k):
        clause = random.sample(variables, random.randint(1, n))
        clauses.append(clause)
    return clauses

def dpll_resolution(clauses):
    def simplify(clauses):
        new_clauses = []
        seen = set()
        for clause in clauses:
            if not clause:
                return None
            if len(clause) == 1:
                seen.add(clause[0])
            else:
                new_clauses.append([x for x in clause if x not in seen and -x not in seen])
        return new_clauses

    def resolve(clauses, literal):
        resolved = []
```

```
        for clause in clauses:
            if literal in clause:
                continue
            if -literal in clause:
                resolved.extend([x for x in clause if x != -literal])
            else:
                resolved.append(clause)
        return resolved

    while True:
        simplified = simplify(clauses)
        if simplified is None:
            return False
        clauses = simplified
        for literal in set(x for clause in clauses for x in clause):
            new_clauses = resolve(clauses, literal)
            if new_clauses is None:
                return True
            clauses = new_clauses

    return len(clauses) == 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    k = random.randint(n // 2, n)
    formula = generate_k_cnf(n, k)

    # Compute Fourier coefficients
    fourier_coeffs = [0] * (1 << n)
    for clause in formula:
        for assignment in range(1 << n):
            if all((assignment & (1 << abs(var) - 1)) != 0 == sign for var, sign in zip(clause, [
                fourier_coeffs[assignment] += 1

    max_coeff = max(abs(coeff) for coeff in fourier_coeffs)

    # Measure resolution proof length
    proof_length = dpll_resolution(formula)

    metric_value = proof_length / (2 ** (n / 2) / max_coeff)
    conjecture_holds = abs(metric_value - 1) <= 0.1
    counterexample = "" if conjecture_holds else "proof_length_mismatch"

    return {
        "metric_name": "Proof Length",
        "metric_value": proof_length,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(5, 3
```

```
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) < 0.2:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"proof_length_mismatch\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34334eed.py", line 110, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_34334eed.py", line 81, in run_trial
    fourier_coeffs = [0] * (1 << n)
                     ~~~~^~~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with MemoryError, preventing evaluation of conjecture validity | next: Optimize memory usage for large n or implement incremental Fourier coefficient calculation

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111282
2	preregistration	ollama_remote	qwen3:8b	0	0	24148
3	novelty	ollama_remote	qwen3:8b	0	0	21161
4	novelty	ollama_remote	qwen3:8b	0	0	15305
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15175
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11565
7	judge	ollama_remote	qwen3:8b	0	0	18869

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 217505 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/3a56915c32e1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3a56915c32e1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3a56915c32e1.tar.gz` (if generated)